

RAPPORT DE STAGE DE FIN D'ETUDES

*5ème Année en Ingénierie Informatique et Réseaux
Option MIAGE*

Mise en place d'une Infrastructure de Données Multi-sources avec Plateforme de Visualisation et Business Intelligence

Réalisé par :

MAKROUM Ahmed

Tuteur (s) :

Encadrant Professionnel : Omar BENTATA.

Encadrant Pédagogique : Badr HIRCHOUA.

Au sein de (Allianz Maroc) :



Année universitaire : 2024/2025

Dédicace

Dédicace

Remerciements

Remerciements

Résumé

resume

Mots-clés :

Abstract

Keywords :

Table des matières

Dédicace	II
Remerciements	III
Résumé	IV
Abstract	V
Liste des sigles et acronymes	XIII
Introduction générale	1
Introduction générale	1
Chapitre 1. Cadre général du stage	3
Introduction	3
1. Présentation de l'organisme d'accueil	3
1.1 Services et domaine d'activité	4
1.2 Clients et collaborateurs	4
2. Cadre du projet	4
2.1 Contexte global du projet	4
2.2 Étude de l'existant	5
2.3 Problématique	5
2.4 Solution proposée	6
3. Langage de modélisation UML	6
4. Méthodologie de travail	7
4.1 Étude comparative des différentes méthodologies	7
4.2 Choix de méthodologie - Scrum	8
4.2.1 Les rôles et les artefacts dans Scrum	8
4.2.2 Justification du choix	9
Chapitre 2. Analyse et spécification des besoins	10
Introduction	10
1. Spécification des besoins	10
1.1 Identification des acteurs	10
1.2 Besoins fonctionnels	10
1.3 Besoins non fonctionnels	11
2. Diagramme global des cas d'utilisation	12
3. Planification et priorisation du projet	12

3.1	Pilotage du projet	12
3.2	Technique de priorisation — MoSCoW	13
4.	Product Backlog	13
5.	Plan de release	15
5.1	Planification détaillée des sprints	16
	Conclusion	16
Chapitre 3. Release 1 — Conception et développement		17
	Introduction	17
1.	Sprint 1 — Boutique-service et Customer-service	17
1.1	Objectif du sprint	17
1.2	Sprint Backlog	17
1.3	Diagramme de cas d'utilisation — Sprint 1	18
1.4	Diagramme de classes — Sprint 1	18
1.4.1	Boutique-service	18
1.4.2	Customer-service	18
1.5	Diagramme de séquence — Créer un client	19
1.6	Interfaces réalisées — Sprint 1	20
2.	Sprint 2 — Catalog-service et Subscription-service	20
2.1	Objectif du sprint	20
2.2	Sprint Backlog	21
2.3	Diagramme de cas d'utilisation — Sprint 2	21
2.4	Diagramme de classes — Sprint 2	21
2.4.1	Catalog-service	21
2.4.2	Subscription-service	22
2.5	Diagramme de séquence — Souscrire un abonnement	23
2.6	Interfaces réalisées — Sprint 2	23
3.	Sprint 3 — Billing-service, Auth, API Gateway et Frontend	23
3.1	Objectif du sprint	23
3.2	Sprint Backlog	24
3.3	Diagramme de cas d'utilisation — Sprint 3	24
3.4	Diagramme de classes — Sprint 3	24
3.4.1	Billing-service	24
3.4.2	Authentication-service	25
3.5	Diagramme de séquence — Générer une facture	26
3.6	Architecture technique de la Release 1	26
3.7	Interfaces réalisées — Sprint 3	27
	Conclusion	27

Chapitre 4. Release 2 — Mise en place de l’infrastructure	28
Introduction	28
1. Sprint 4 — Containerisation et déploiement local	28
1.1 Objectif du sprint	28
1.2 Sprint Backlog	28
1.3 Containerisation avec Docker	28
1.4 Orchestration locale avec Docker Compose	29
1.5 Déploiement sur Minikube	29
2. Sprint 5 — Déploiement sur le cloud AWS	30
2.1 Objectif du sprint	30
2.2 Sprint Backlog	30
2.3 Architecture de l’infrastructure AWS	30
2.4 Provisionnement du cluster EKS	31
2.5 Configuration d’Amazon RDS	31
Conclusion	32
 Chapitre 5. Release 3 — Pipeline CI/CD	 33
Introduction	33
1. Sprint 6 — Pipeline CI/CD et déploiement continu	33
1.1 Objectif du sprint	33
1.2 Sprint Backlog	33
1.3 Architecture du pipeline CI/CD	33
1.4 Serveur Jenkins	34
1.5 Pipeline Jenkinsfile	35
1.6 Analyse de qualité — SonarQube	35
1.7 Scan de sécurité — Trivy	36
1.8 Déploiement GitOps avec ArgoCD	36
1.9 Packaging avec Helm	36
Conclusion	36
 Chapitre 6. Release 4 — Monitoring et supervision	 37
Introduction	37
1. Sprint 7 — Prometheus, Grafana et dashboards	37
1.1 Objectif du sprint	37
1.2 Sprint Backlog	37
1.3 Architecture de monitoring	37
1.4 Déploiement de Prometheus	38
1.5 Dashboards Grafana	38
Conclusion	39

Conclusion générale	40
Webographie	43

Liste des figures

1.1	Logo de la société d'accueil	3
1.2	Les clients de Billcom Consulting	4
1.3	Interface de gestion client de la solution Telkoware	5
1.4	La méthodologie Scrum	8
2.1	Diagramme global des cas d'utilisation	12
3.1	Diagramme de cas d'utilisation du Sprint 1	18
3.2	Diagramme de classes du Boutique-service	18
3.3	Diagramme de classes du Customer-service	19
3.4	Diagramme de séquence : création d'un client	19
3.5	Interface de gestion des boutiques	20
3.6	Interface de gestion des clients	20
3.7	Diagramme de cas d'utilisation du Sprint 2	21
3.8	Diagramme de classes du Catalog-service	22
3.9	Diagramme de classes du Subscription-service	22
3.10	Diagramme de séquence : souscription d'un abonnement	23
3.11	Interface du catalogue des offres	23
3.12	Diagramme de cas d'utilisation du Sprint 3	24
3.13	Diagramme de classes du Billing-service	25
3.14	Diagramme de classes de l'Authentication-service	25
3.15	Diagramme de séquence : génération d'une facture	26
3.16	Architecture technique des microservices	26
3.17	Interface de gestion des factures	27
3.18	Interface d'authentification	27
4.1	Pods déployés sur le cluster Minikube	29
4.2	Architecture de l'infrastructure AWS	31
4.3	Cluster EKS sur la console AWS	31
5.1	Architecture du pipeline CI/CD	34
5.2	Pipeline Jenkins avec les différentes étapes	35
5.3	Résultats d'analyse SonarQube	35
5.4	Dashboard ArgoCD	36

6.1	Architecture de la stack de monitoring	38
6.2	Dashboard Grafana de supervision	39
6.3	Targets Prometheus	39

Liste des tableaux

1.1	Fiche d'identification de l'entreprise d'accueil	3
1.2	Tableau comparatif entre les méthodologies	7
1.3	Avantages et inconvénients des méthodologies	7
2.1	Besoins fonctionnels par acteur	11
2.2	Équipe Scrum du projet	13
2.3	Product Backlog du projet	14
2.4	Plan de release du projet	15
2.5	Planification des sprints avec objectifs (Sprint Goals)	16
3.1	Sprint Backlog — Sprint 1	17
3.2	Sprint Backlog — Sprint 2	21
3.3	Sprint Backlog — Sprint 3	24
4.1	Sprint Backlog — Sprint 4	28
4.2	Sprint Backlog — Sprint 5	30
5.1	Sprint Backlog — Sprint 6	33
6.1	Sprint Backlog — Sprint 7	37

Liste des abbréviations

PoC	Proof of Concept
PGSQL	PostgresSQL : Système de gestion de base de données relationnelle
Spark	Apache Spark (moteur de traitement distribué de données)
SQL	Structured Query Language

Introduction générale

Introduction générale

Dans le domaine de l'ingénierie logicielle contemporaine, la conception et le déploiement d'applications d'entreprise connaissent une mutation technologique profonde. Pendant de nombreuses années, les architectures monolithiques ont constitué le standard pour développer des systèmes d'information complets. Bien que ce modèle de conception centralisé ait fait ses preuves pour des projets de complexité modérée, il montre rapidement ses limites dans les environnements soumis à de fortes contraintes de pérennité : les cycles de déploiement sont lents, la dette technique s'accumule, et une défaillance locale peut entraîner l'indisponibilité totale du système.

C'est pour répondre à cette rigidité structurelle que le paradigme des microservices s'est imposé, complété par l'émergence incontournable de la culture DevOps. Cette nouvelle approche ne se limite plus à écrire du code ; elle redéfinit l'intégralité du cycle de vie du logiciel en prônant l'automatisation, la containerisation, et l'orchestration dynamique sur le Cloud.

C'est dans ce contexte d'innovation technologique que s'inscrit le présent projet de fin d'études, réalisé au sein de la société Billcom Consulting. L'objectif principal est de concevoir, développer, et automatiser le déploiement d'une plateforme métier orientée microservices (ici appliquée à un cas d'usage de facturation et de gestion). Au-delà de la réingénierie logicielle, l'enjeu majeur de ce projet réside dans la mise en œuvre d'une chaîne DevSecOps complète : de l'intégration continue jusqu'au déploiement GitOps sur un cluster Kubernetes managé, avec une supervision en temps réel.

Pour structurer ce travail, nous avons adopté la méthodologie agile Scrum, articulée autour de quatre "Releases" majeures. Le présent rapport s'organise ainsi en six chapitres distincts :

Le **premier chapitre** dresse le cadre général du projet, en présentant l'organisme d'accueil, le contexte technologique, la problématique rencontrée ainsi que la solution proposée et la méthodologie de travail adoptée.

Le **deuxième chapitre** est consacré à l'analyse et la spécification des besoins. Il identifie les acteurs du système, détaille les fonctionnalités attendues sous forme de cas d'utilisation et présente le *Product Backlog* ainsi que le plan de release.

Le **troisième chapitre** marque le début de la réalisation (Release 1). Il détaille la conception et le développement de l'ensemble des microservices (gestion des clients, catalogue, abonnements et facturation) et la sécurisation des échanges.

Le **quatrième chapitre** (Release 2) aborde la mise en place de l'infrastructure logicielle. Il explique la containerisation des applications via Docker et leur déploiement sur les envi-

ronnements Kubernetes, allant d'un cluster local (Minikube) vers le cloud AWS (Amazon EKS).

Le **cinquième chapitre** (Release 3) est dédié à l'automatisation. Il décrit la mise en place du pipeline CI/CD avec Jenkins, l'intégration des outils d'analyse de qualité et de sécurité (SonarQube, Trivy), et le déploiement continu via ArgoCD.

Enfin, le **sixième chapitre** (Release 4) expose la mise en place d'une stack complète de monitoring et de supervision à l'aide de Prometheus et Grafana, garantissant ainsi la fiabilité de la solution en production.

Chapitre 1. Cadre général du stage

Introduction

Dans ce chapitre, on va présenter le cadre général de notre projet. on va d'abord présenter l'entreprise d'accueil, ensuite on va définir le cadre de ce projet de fin d'études à travers l'analyse de l'existant et la présentation de la problématique ainsi que la solution proposée, et enfin on va détailler la méthodologie de conception adoptée.

1. Présentation de l'organisme d'accueil

L'entreprise d'accueil est Billcom Consulting, une société de services et de conseil fondée en 2007, Billcom est spécialisée dans l'accompagnement des opérateurs de télécommunications par l'intégration des solutions BSS dont la gestion de la relation client et de la facturation, ainsi que la migration de données. Basée en Tunisie, l'entreprise exerce ses activités auprès de clients situés à l'international, notamment en Afrique, au Moyen-Orient et en Europe.



FIGURE 1.1 – Logo de la société d'accueil

Raison sociale	Billcom Consulting
Forme juridique	SARL
Adresse	Rue des Orangers Hammam Sousse
Téléphone	58597507
E-mail	contact@billcom-consulting.com
Secteur d'activité	Conseil et intégration de solutions télécoms

TABLE 1.1 – Fiche d'identification de l'entreprise d'accueil

1.1 Services et domaine d'activité

Afin de mieux présenter le domaine d'activité de Billcom Consulting, on va citer les services et les solutions offerts par cette société :

- **Inégration des solutions de facturation et gestion client** : déployer et maintenir des systèmes de gestion des abonnés (Customer Care) et des systèmes de facturation, incluant notamment les solutions technologiques de type BSS.
- **Business Consulting** : accompagner les clients dans l'analyse de leurs besoins, l'étude de leurs processus et le choix de solutions techniques et fonctionnelles cohérentes avec leurs objectifs.
- **Support technique** : assurer le support technique, la maintenance des solutions en place et l'assistance aux utilisateurs afin de garantir la continuité de service.

1.2 Clients et collaborateurs

Billcom Consulting s'est bien positionné sur le marché en fidélisant de nombreux clients. Parmi ceux-ci, on cite :



FIGURE 1.2 – Les clients de Billcom Consulting

2. Cadre du projet

2.1 Contexte global du projet

Les opérateurs télécoms s'appuient sur des systèmes d'information complexes pour gérer l'ensemble du cycle de vie de leurs offres et services. Ces systèmes couvrent plusieurs processus métiers essentiels, notamment la gestion des clients, l'abonnement aux offres, le suivi de la consommation, la facturation et le recouvrement.

Avec l'évolution des besoins (nombre d'utilisateurs, diversité des offres, intégration avec d'autres systèmes), ces plateformes deviennent de plus en plus complexes à maintenir et à faire évoluer.

2.2 Étude de l'existant

Afin de justifier pleinement les choix architecturaux et technologiques opérés dans la création de notre application de gestion des services commerciaux, il est essentiel d'analyser les systèmes actuellement utilisés chez les opérateurs télécoms.

Pour éviter de développer ces systèmes complexes en interne, la majorité des opérateurs se tourne vers des éditeurs logiciels spécialisés proposant des suites BSS prêtes à l'emploi. Dans cette approche, l'ensemble des modules décrits précédemment (CRM, Catalogue, Boutiques, Consommation, Facturation, Paiement) est compilé et regroupé au sein d'une seule et même application massive.

On cite quelques solutions similaires :

- **Telkaware**
- **Amdocs Kenan**

FIGURE 1.3 – Interface de gestion client de la solution Telkaware

2.3 Problématique

Les équipes informatiques se retrouvent confrontées à plusieurs défis majeurs : la difficulté à livrer de nouvelles fonctionnalités rapidement sans perturber les opérations en cours et la difficulté de faire évoluer un composant du système de manière indépendante. Ces fonctionnalités sont souvent regroupées au sein de plateformes monolithiques centralisées.

Cette approche visait principalement à simplifier l'intégration initiale et à garantir la cohérence des données entre les différents modules métiers.

2.4 Solution proposée

Afin de répondre aux problématiques identifiées, une solution basée sur une architecture modulaire est proposée. Elle s'inspire des systèmes utilisés dans le domaine des télécommunications (BSS), tout en restant adaptée à un contexte de projet de fin d'études.

La solution consiste à développer un système organisé en modules indépendants couvrant les principales fonctionnalités métier, notamment la gestion des clients, la gestion des services et des abonnements, la facturation, ainsi que le suivi via un tableau de bord. Dans ce contexte, on a pensé à des interfaces simples et des actions faciles à utiliser en réduisant la navigation.

Les enjeux de ce projet sont l'automatisation, l'intégration du code ainsi que la livraison continue. L'objectif est de mettre en place un pipeline CI/CD, puis le déploiement de l'application dans un cluster Kubernetes, ainsi que l'implémentation d'une partie de surveillance (monitoring) de l'application en environnement de production. Ce projet s'inscrit dans une démarche DevOps, il constitue quatre phases principales :

- **Développement de l'application**
- **Intégration continue / Livraison continue**
- **Déploiement de l'application dans un cluster Kubernetes**
- **Monitoring et alerting**

3. Langage de modélisation UML

UML signifie "Unified Modeling Language" ou "Langage de Modélisation Unifié" en français. C'est un langage de modélisation graphique utilisé pour visualiser, spécifier, construire et documenter les systèmes logiciels. UML fournit un ensemble de notations et de diagrammes standardisés qui permettent de présenter le logiciel.

Il est composé de trois types de diagrammes (diagrammes statiques, diagrammes de comportement et diagrammes dynamiques), chacun ayant une fonction spécifique pour représenter différents aspects du système. Parmi les types de diagrammes les plus couramment utilisés, on trouve :

Les diagrammes de classes, les diagrammes de cas d'utilisation, les diagrammes de séquence et les diagrammes d'activités.

4. Méthodologie de travail

Pour mener à bien le développement d'un projet informatique, il est très important de définir la méthode à suivre au cours du travail ainsi que les tâches à effectuer par la suite pour garantir un niveau de qualité acceptable et éviter les délais.

4.1 Étude comparative des différentes méthodologies

Le tableau suivant résume les différences entre l'approche classique et l'approche agile.

Thème	Approche classique	Approche agile
Cycle de vie	Séquentiel, en cascade.	Itératif et incrémental.
Planification	Définie dès le départ.	Adaptative et évolutive.
Documentation	Importante et détaillée.	Réduite au strict nécessaire.
Équipe	Rôles spécialisés, pilotage centralisé.	Équipe autonome et collaborative.
Changement	Résistance et gestion lourde.	Changement fréquent.
Validation	En fin de projet.	Continue, au fil des itérations.

TABLE 1.2 – Tableau comparatif entre les méthodologies

Après une étude comparative des méthodologies , présentées précédemment, nous avons établi un tableau récapitulatif, illustré par le tableau 1.3 , qui contient les avantages et les inconvénients de chaque méthode

Méthodologie	Avantages	Inconvénients
Classique	- Simple à mettre en oeuvre - Adaptée aux projets stables	- Peu flexible - Spécifications figées dès le départ - Non adaptable à tout type de projet
Agile	- Livraisons rapides - Meilleure adaptation aux besoins clients - Améliore la qualité d'exécution	- Forte implication requise - Maîtrise de Scrum nécessaire - Non adaptable à tout type de projet

TABLE 1.3 – Avantages et inconvénients des méthodologies

4.2 Choix de méthodologie - Scrum

La méthode Scrum est une approche agile qu'on a adopté afin de garantir la flexibilité, l'efficacité et l'adaptabilité. Contrairement aux méthodes traditionnelles comme la méthodologie en V ou le modèle en cascade, l'agilité repose sur des cycles courts et itératifs, mieux adaptés aux projets évolutifs. Elle suppose que le développement d'un logiciel n'est pas prévisible et ne peut donc pas suivre de processus défini. Ainsi, le résultat du projet n'est pas déterminé au départ, il dépend des retours du client permettant de s'adapter aux imprévus et de maintenir une communication constante avec les parties prenantes.

La figure suivante présente le schéma décrivant les principes de la méthodologie Scrum. Le principe de base de Scrum est de diviser le projet sur un ensemble d'itérations à réaliser, appelées Sprints. Chaque Sprint possède des buts à atteindre, à partir desquels sont définies les fonctionnalités à implémenter. Ces fonctionnalités sont définies dans un backlog de produit sous forme de User Stories, qui sont des descriptions courtes et simples d'une fonctionnalité du point de vue de l'utilisateur final. Un sprint aboutit toujours à la livraison d'une version améliorée et partiellement fonctionnelle du produit appelée MVP.

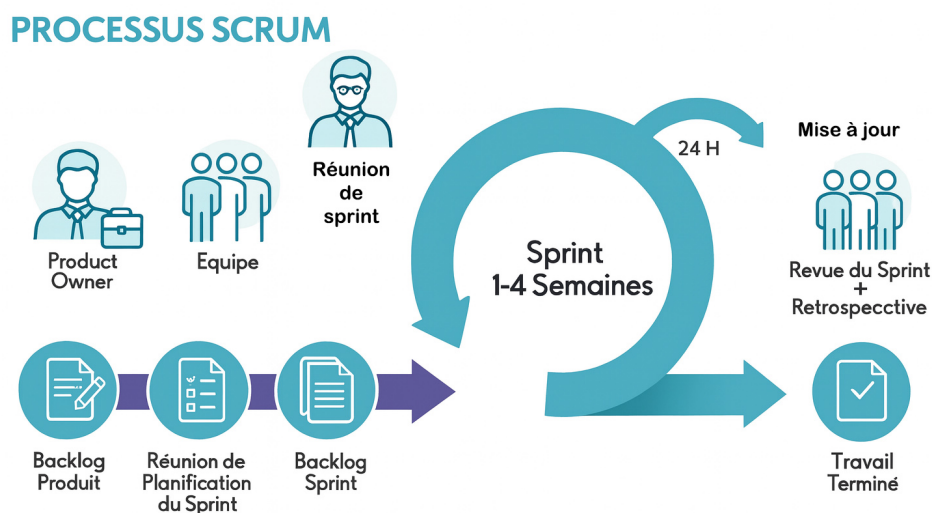


FIGURE 1.4 – La méthodologie Scrum

4.2.1 Les rôles et les artefacts dans Scrum

Les trois rôles principaux dans Scrum sont :

- **Product Owner** : il représente le client et les parties prenantes. Il est responsable de maximiser la valeur du produit il en porte la vision métier. Il définit la feuille de route et gère le *Product Backlog* : contenu, disponibilité et ordre de priorité des éléments.

- **Scrum Master** : il veille au respect du cadre Scrum et facilite son application au quotidien. Son rôle est d'accompagner le Product Owner et l'équipe de développement, de lever les obstacles et de favoriser l'amélioration continue. Ce n'est pas un chef de projet, même si les deux rôles peuvent parfois être cumulés selon le contexte.
- **Équipe de développement** : réalise un incrément livrable du produit à chaque Sprint selon les priorités définies. Elle est pluridisciplinaire et peut regrouper différents profils techniques, comme des développeurs, des architectes et des administrateurs de bases de données.

Les artefacts principaux de Scrum sont :

- **Product Backlog** : Une liste structurée de tous les requis, besoins du client et spécifications du produit. C'est le Product Owner qui est chargé de sa mise à jour tout au long du projet.
- **Sprint Backlog** : Un schéma précis des actions à réaliser au cours du Sprint. Il est conçu en début de chaque Sprint à partir du product backlog. Cela permet à l'équipe de surveiller l'avancement et de répondre aux objectifs établis par le Product Owner.

4.2.2 Justification du choix

Le choix de la méthodologie Scrum fournit des avantages pour les clients, Scrum permet au client de tester et de valider les fonctionnalités au fur et à mesure de leur développement ce qui permet d'obtenir un retour d'information rapide et de s'assurer que le produit final correspond aux attentes du client permettant de minimiser les problèmes de compréhension de la conception. En plus, le client peut ajouter ou modifier les fonctionnalités pendant le sprint planning ou le sprint review. Ainsi, Scrum assure la qualité du produit livré.

Conclusion

Dans ce chapitre, on a présenté l'entreprise d'accueil, le cadre général de notre projet. Par la suite on a proposé la solution adoptée suite à l'étude de l'existant. Enfin, on a détaillé la méthodologie de travail à suivre pour mener à bien le projet.

Chapitre 2. Analyse et spécification des besoins

Introduction

L'étape de spécification des besoins constitue la base de départ de ce travail. Ce chapitre comprend l'analyse des besoins fonctionnels et non fonctionnels, l'identification des acteurs, la définition du backlog produit et la planification des releases et des sprints selon la méthodologie Scrum.

1. Spécification des besoins

1.1 Identification des acteurs

Un acteur représente une entité externe qui interagit avec le système. L'analyse des besoins métier et de l'architecture de l'application a permis d'identifier trois acteurs principaux :

- **Administrateur** : Supervise l'ensemble de la plateforme. Il gère les utilisateurs et leurs rôles, administre le catalogue des offres et des services télécom, consulte l'historique de facturation et dispose d'un accès complet à toutes les fonctionnalités du système.
- **Agent commercial** : Représente l'utilisateur opérationnel. Il crée et gère les fiches clients, souscrit des abonnements, consulte le catalogue des offres disponibles et effectue les opérations de vente au niveau de la boutique.
- **Responsable boutique** : Gère un point de vente. Il administre le stock de cartes SIM, supervise l'équipe commerciale, met à jour les informations des clients rattachés à sa boutique et consulte les abonnements au niveau de son point de vente.

1.2 Besoins fonctionnels

Les besoins fonctionnels représentent les attentes de chaque acteur vis-à-vis des modules à développer.

Le tableau suivant détaille les besoins fonctionnels identifiés, regroupés par acteur du système :

Acteur	Besoin fonctionnel
Administrateur	Gérer les utilisateurs et attribuer les rôles.
	Gérer les offres commerciales et les services télécom.
	Consulter l'historique détaillé de facturation des clients.
	Gérer les boutiques.
	Gérer le cycle de vie des abonnements.
Agent commercial	Créer un nouveau client avec validation (CIN, téléphone).
	Consulter et rechercher le profil d'un client.
	Consulter le catalogue des offres disponibles.
	Souscrire un abonnement liant un client à une offre.
	Générer et exporter des factures en PDF.
	Envoyer une facture par email au client.
Responsable boutique	Consulter la liste et les détails des boutiques.
	Gérer le stock de cartes SIM (ajout, activation, suspension).
	Gérer l'équipe commerciale de la boutique.
	Mettre à jour les informations des clients rattachés.
	Consulter les abonnements au niveau du point de vente.

TABLE 2.1 – Besoins fonctionnels par acteur

1.3 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité, de performance et de sécurité que le système doit respecter.

- **Scalabilité** : Chaque microservice doit pouvoir être mis à l'échelle individuellement en fonction de sa charge, sans impacter les autres services.
- **Disponibilité** : La plateforme doit garantir une haute disponibilité. La défaillance d'un service ne doit pas entraîner l'arrêt du système.
- **Sécurité** : L'accès doit être sécurisé par l'authentification JWT et le contrôle d'accès basé sur les rôles (RBAC). Les mots de passe sont chiffrés.

- **Performance** : Les microservices doivent répondre rapidement aux requêtes, avec des temps de réponse optimisés.
- **Ergonomie** : L'interface doit être intuitive, avec un design minimaliste facilitant la navigation.

2. Diagramme global des cas d'utilisation

Le diagramme de cas d'utilisation global ci-dessous présente une vue d'ensemble des interactions entre les trois acteurs et les principales fonctionnalités du système.



FIGURE 2.1 – Diagramme global des cas d'utilisation

Les principales interactions sont :

- L'**administrateur** gère le catalogue des services, les offres commerciales et les utilisateurs.
- L'**agent commercial** gère les clients, souscrit des abonnements et traite la facturation.
- Le **responsable boutique** gère le stock de cartes SIM, supervise l'équipe et consulte les données de sa boutique.

3. Planification et priorisation du projet

3.1 Pilotage du projet

Dans le contexte de la méthodologie Scrum, l'équipe se structure autour de trois rôles :

Rôle	Responsable
Product Owner	Mr. Mohamed Louragini
Scrum Master	Mr. Adnen Rouatbi
Équipe de développement	Ghassen Chouchen

TABLE 2.2 – Équipe Scrum du projet

3.2 Technique de priorisation — MoSCoW

Le backlog produit est une liste ordonnée de tous les éléments sources de valeur pour le client. Pour assurer une livraison rapide de valeur, il est essentiel de prioriser les éléments du backlog.

Comme technique de priorisation, on a adopté **la méthode MoSCoW** :

- **M – Must have** : Éléments essentiels pour le projet.
- **S – Should have** : Éléments importants mais pas critiques.
- **C – Could have** : Éléments souhaitables mais non essentiels.
- **W – Won’t have** : Éléments exclus du périmètre actuel.

4. Product Backlog

Le *backlog produit* regroupe l’ensemble des fonctionnalités à développer, exprimées sous forme de *User Stories* pour les besoins fonctionnels et d’*Enablers* pour les tâches techniques. Ces éléments sont ordonnés et priorisés selon la méthode MoSCoW.

Le tableau 2.3 présente le backlog produit global du projet.

Ordre	Type	Thème	Description	Priorité	Points
1	US	Boutiques	ETQ agent, je veux créer et consulter les boutiques afin de gérer les points de vente.	Must	5
2	US	Clients	ETQ agent, je veux créer un client afin de pouvoir lui associer un abonnement.	Must	8
3	US	Clients	ETQ agent, je veux modifier un client afin de garder ses informations à jour.	Must	3
4	US	Catalogue	ETQ administrateur, je veux gérer le catalogue afin de structurer les services télécom.	Must	5

Ordre	Type	Thème	Description	Priorité	Points
5	US	Offres	ETQ administrateur, je veux créer des offres afin de les proposer aux clients.	Must	8
6	US	Abonnements	ETQ agent, je veux souscrire un client à une offre afin de lui fournir un service.	Must	13
7	US	SIM	ETQ responsable, je veux gérer le stock SIM afin d'activer les lignes des clients.	Must	8
8	US	Auth	ETQ utilisateur, je veux m'authentifier afin d'accéder au système de façon sécurisée.	Must	5
9	Enabler	Architecture	Mise en place de l'API Gateway et d'Eureka pour le routage et la découverte des services.	Must	8
10	US	Facturation	ETQ agent, je veux générer des factures afin de facturer les abonnements.	Must	13
11	US	Facturation	ETQ agent, je veux exporter une facture en PDF afin de la fournir au client.	Must	5
12	US	Notification	ETQ agent, je veux envoyer les factures par email afin de notifier le client.	Should	5
13	US	Interface	ETQ agent, je veux une interface web afin de gérer facilement les opérations.	Must	13
14	Enabler	Déploiement	Création des Dockerfiles et déploiement local des services sur Minikube.	Must	8
15	Enabler	Cloud	Provisionnement de l'infrastructure AWS (EKS, RDS, ECR).	Must	13
16	Enabler	CI/CD	Mise en place du pipeline Jenkins (build, SonarQube, Trivy, push ECR).	Must	13
17	Enabler	GitOps	Configuration d'ArgoCD et Helm pour le déploiement continu.	Must	8
18	Enabler	Monitoring	Déploiement de Prometheus et Grafana pour la supervision.	Should	5

TABLE 2.3 – Product Backlog du projet

Légende : US = User Story (besoin fonctionnel), Enabler = Tâche technique.

5. Plan de release

Le projet s'étend sur 15 semaines et est organisé en 4 releases contenant au total 7 sprints de 2 semaines chacun, précédés d'une semaine de cadrage et d'analyse.

Release	Sprint	Objectif
0	<i>Cadrage et analyse</i>	Analyse des besoins et planification
1	Sprint 1	Mise en place des fondations métier (Boutique-service, Customer-service)
	Sprint 2	Gestion du catalogue et des abonnements (Catalog-service, Subscription-service)
	Sprint 3	Facturation, sécurisation et interface (Billing-service, Auth, API Gateway, Frontend)
2	Sprint 4	Containerisation et déploiement local (Dockerfiles, Minikube, Manifests K8s)
	Sprint 5	Migration vers le Cloud AWS (Amazon EKS, RDS, ECR)
3	Sprint 6	Automatisation CI/CD et GitOps (Jenkins, SonarQube, Trivy, Helm, ArgoCD)
4	Sprint 7	Supervision analytique (Prometheus, Grafana, dashboards)

TABLE 2.4 – Plan de release du projet

5.1 Planification détaillée des sprints

Sprint	Rel.	Sem.	Titre du Sprint	Modules clés
Sprint 1	R1	2–3	Mise en place des fondations métier	Boutique-service, Customer-service
Sprint 2	R1	4–5	Gestion du catalogue et des abonnements	Catalog-service, Subscription-service
Sprint 3	R1	6–7	Facturation, sécurisation et interface	Billing-service, Auth, API Gateway, Frontend
Sprint 4	R2	8–9	Containerisation et déploiement local	Dockerfiles, Minikube, Manifests K8s
Sprint 5	R2	10–11	Migration vers le Cloud AWS	Amazon EKS, RDS, ECR
Sprint 6	R3	12–13	Automatisation CI/CD et GitOps	Jenkins, SonarQube, Trivy, Helm, ArgoCD
Sprint 7	R4	14–15	Supervision analytique	Prometheus, Grafana

TABLE 2.5 – Planification des sprints avec objectifs (Sprint Goals)

Conclusion

Dans ce chapitre, nous avons identifié les acteurs du système, spécifié les besoins fonctionnels et non fonctionnels, élaboré le backlog produit priorisé et défini le plan de release. Le chapitre suivant présentera la première release consacrée à la conception et au développement des microservices métier.

Chapitre 3. Release 1 — Conception et développement

Introduction

Cette release constitue le cœur métier de la plateforme de facturation. Elle couvre le développement de l'ensemble des microservices fonctionnels répartis sur trois sprints. Chaque sprint est accompagné de sa modélisation UML (diagrammes de cas d'utilisation, de classes et de séquence).

1. Sprint 1 — Boutique-service et Customer-service

1.1 Objectif du sprint

Ce sprint vise à mettre en place les deux premiers microservices fondamentaux : la gestion des boutiques (points de vente) et la gestion des clients.

1.2 Sprint Backlog

ID	Tâche	Priorité	Points
1.1	CRUD Boutique (création, consultation, modification)	Must	5
1.2	CRUD Client avec validation CIN (8 chiffres) et téléphone (8 chiffres)	Must	8
1.3	Modification et suspension d'un client	Must	3
1.4	Publication d'événements client via le bus de messages	Should	3

TABLE 3.1 – Sprint Backlog — Sprint 1

1.3 Diagramme de cas d'utilisation — Sprint 1

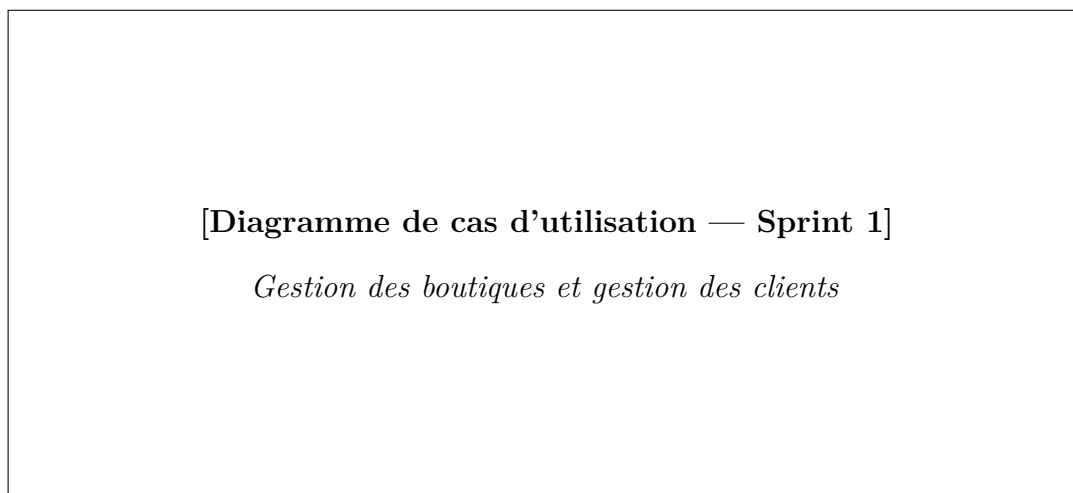


FIGURE 3.1 – Diagramme de cas d'utilisation du Sprint 1

1.4 Diagramme de classes — Sprint 1

1.4.1 Boutique-service

Le microservice Boutique gère les points de vente et le stock de cartes SIM. Les principales entités sont :

- **Boutique** : Représente un point de vente avec un nom, une adresse, un responsable et un statut (ACTIVE, INACTIVE).
- **SimCard** : Représente une carte SIM avec un numéro ICCID, un statut (AVAILABLE, ASSIGNED, SUSPENDED) et une référence au client assigné.

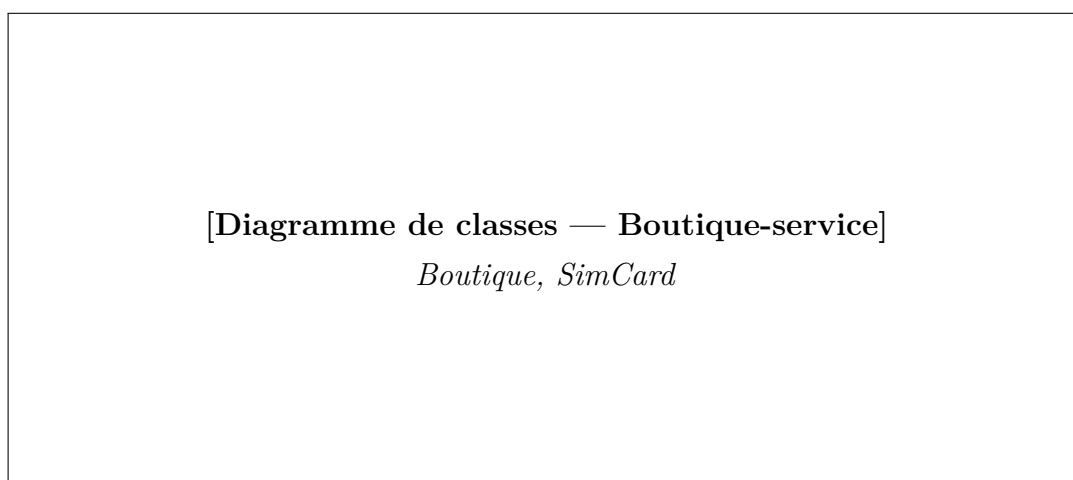


FIGURE 3.2 – Diagramme de classes du Boutique-service

1.4.2 Customer-service

Le microservice Customer gère le cycle de vie des clients. L'entité principale est :

- **Customer** : Contient les informations personnelles du client (nom, prénom, email, CIN, téléphone, adresse), une référence unique (format CLT-ANNÉE-UUID), un statut (ACTIVE, SUSPENDED, TERMINATED) et la référence de la boutique rattachée.

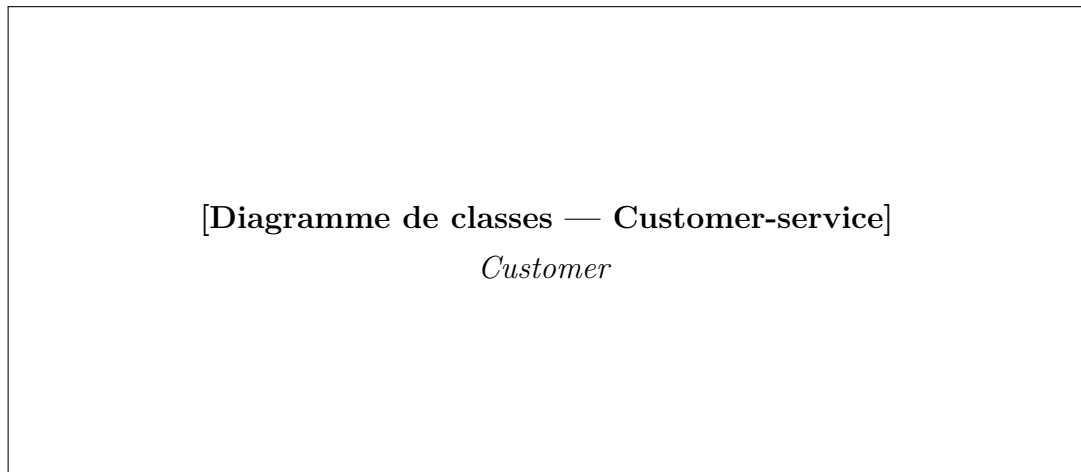


FIGURE 3.3 – Diagramme de classes du Customer-service

1.5 Diagramme de séquence — Créer un client

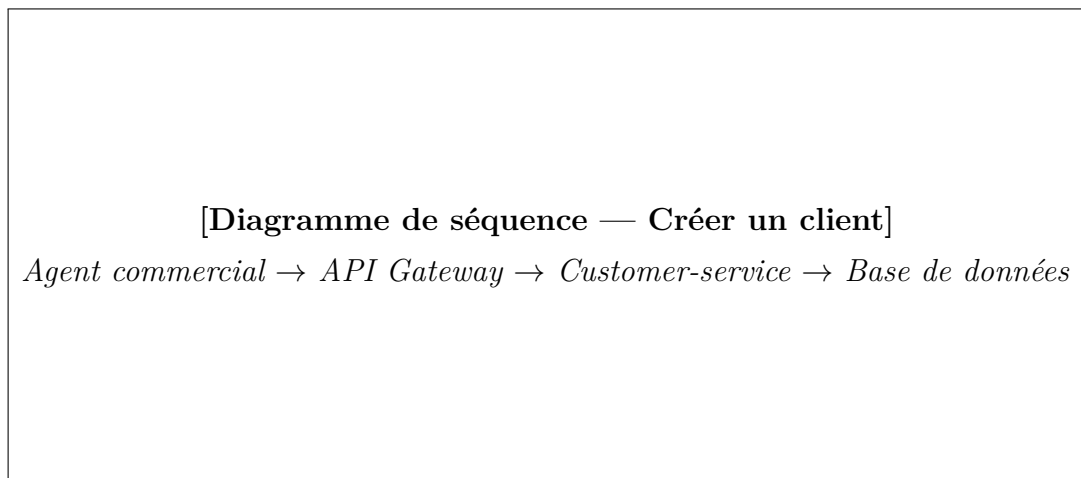


FIGURE 3.4 – Diagramme de séquence : création d'un client

1.6 Interfaces réalisées — Sprint 1

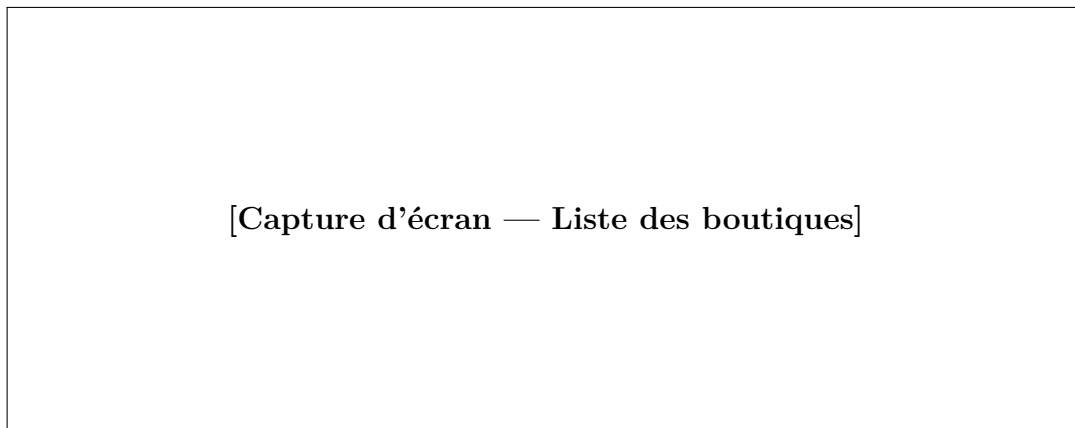


FIGURE 3.5 – Interface de gestion des boutiques

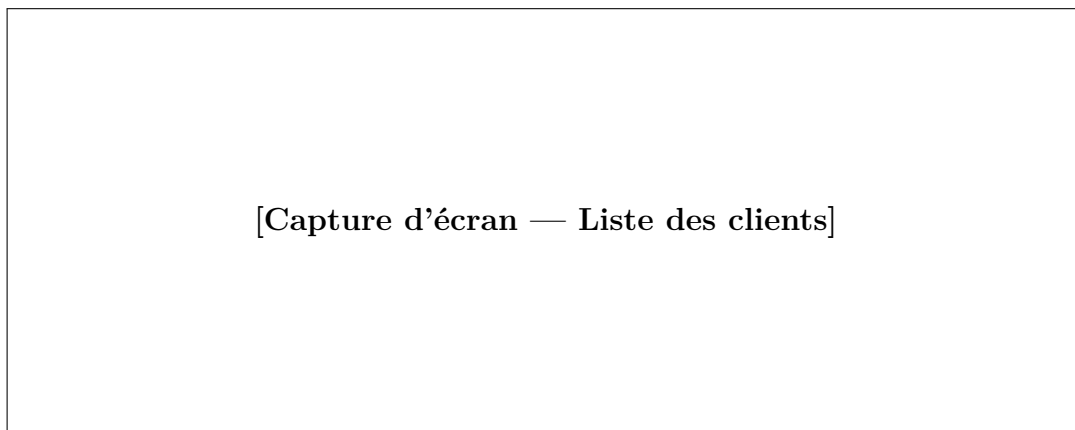


FIGURE 3.6 – Interface de gestion des clients

2. Sprint 2 — Catalog-service et Subscription-service

2.1 Objectif du sprint

Ce sprint vise à développer le catalogue des services télécom et le mécanisme de souscription aux offres commerciales.

2.2 Sprint Backlog

ID	Tâche	Priorité	Points
2.1	CRUD Service télécom (voix, data, SMS)	Must	5
2.2	CRUD Offre commerciale (regroupement de services)	Must	8
2.3	Souscription d'un client à une offre (création d'abonnement)	Must	13
2.4	Gestion du cycle de vie d'un abonnement (activation, suspension, résiliation)	Must	5

TABLE 3.2 – Sprint Backlog — Sprint 2

2.3 Diagramme de cas d'utilisation — Sprint 2

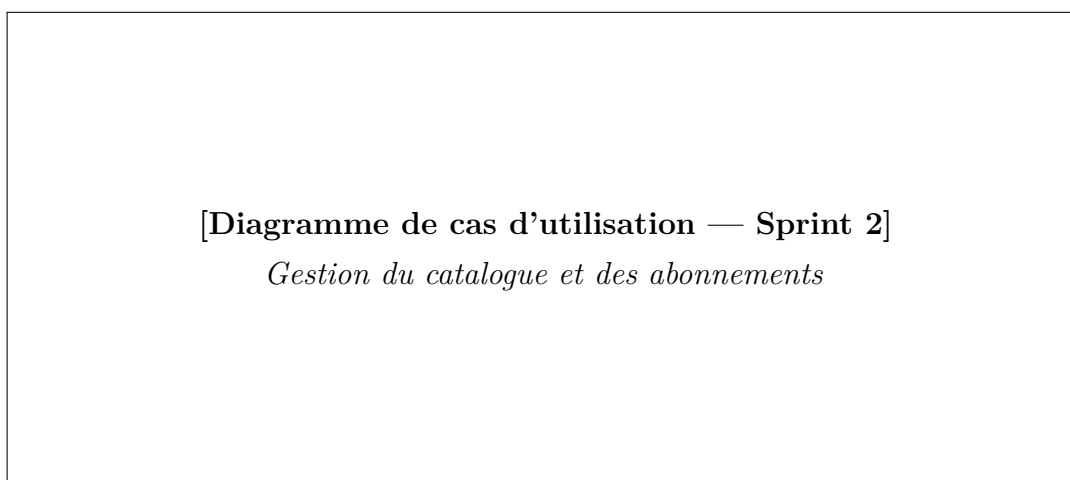


FIGURE 3.7 – Diagramme de cas d'utilisation du Sprint 2

2.4 Diagramme de classes — Sprint 2

2.4.1 Catalog-service

Le microservice Catalog gère les services télécom et les offres commerciales :

- **ServiceEntity** : Représente un service unitaire (voix, data, SMS) avec un code, un nom, une description et un tarif.
- **Offre** : Regroupe un ensemble de services en une offre commerciale, avec un nom, un prix, une durée de validité et un statut (ACTIVE, DISCONTINUED).

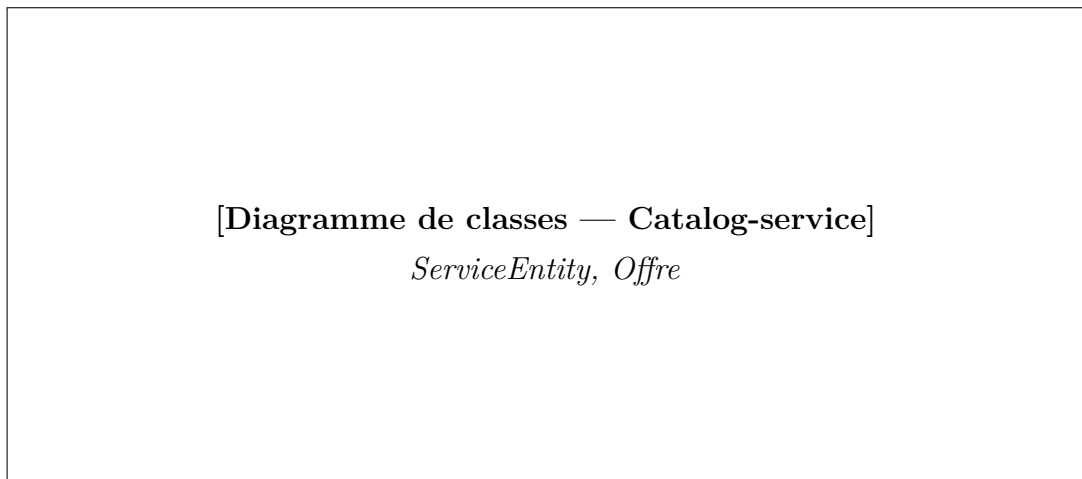


FIGURE 3.8 – Diagramme de classes du Catalog-service

2.4.2 Subscription-service

Le microservice Subscription gère les abonnements :

- **Abonnement** : Lie un client à une offre avec une date de début, une date de fin, un statut (ACTIVE, SUSPENDED, TERMINATED) et une fréquence de facturation.

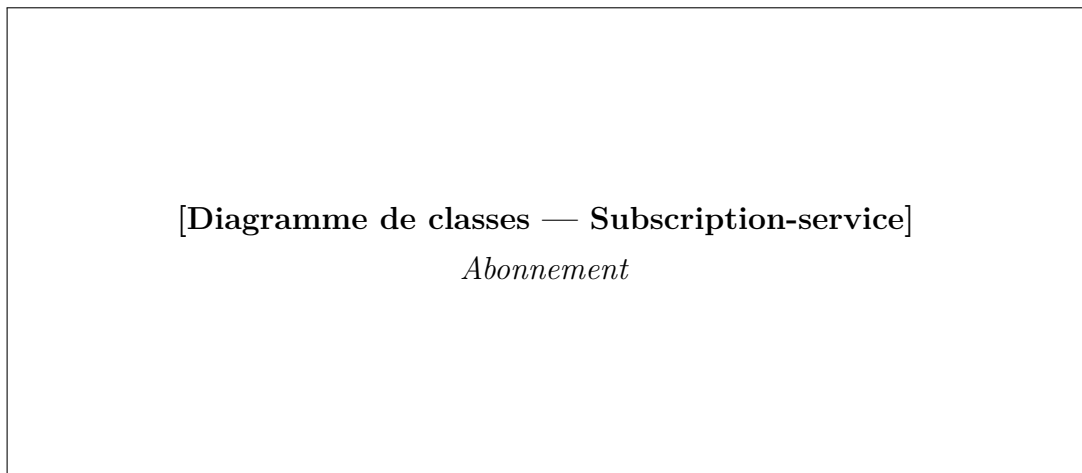


FIGURE 3.9 – Diagramme de classes du Subscription-service

2.5 Diagramme de séquence — Souscrire un abonnement

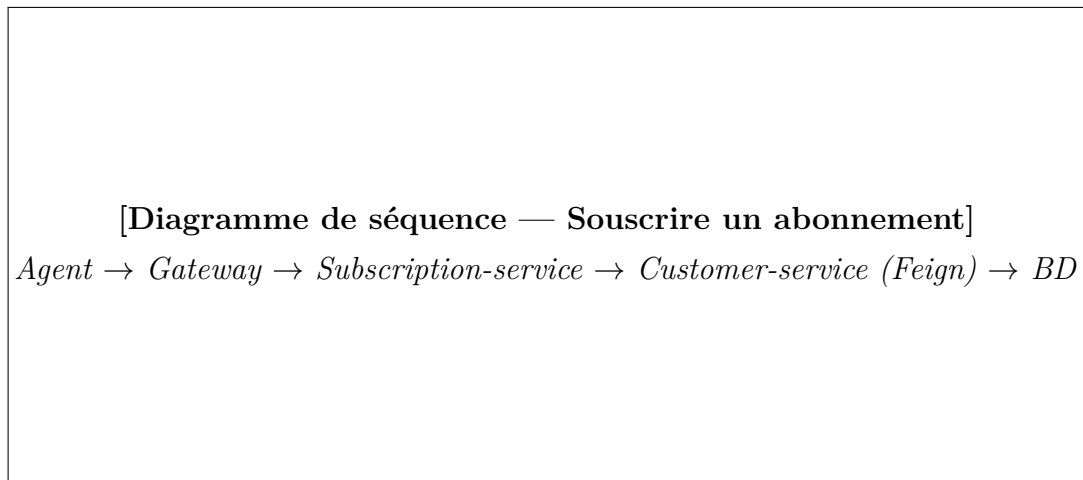


FIGURE 3.10 – Diagramme de séquence : souscription d’un abonnement

2.6 Interfaces réalisées — Sprint 2

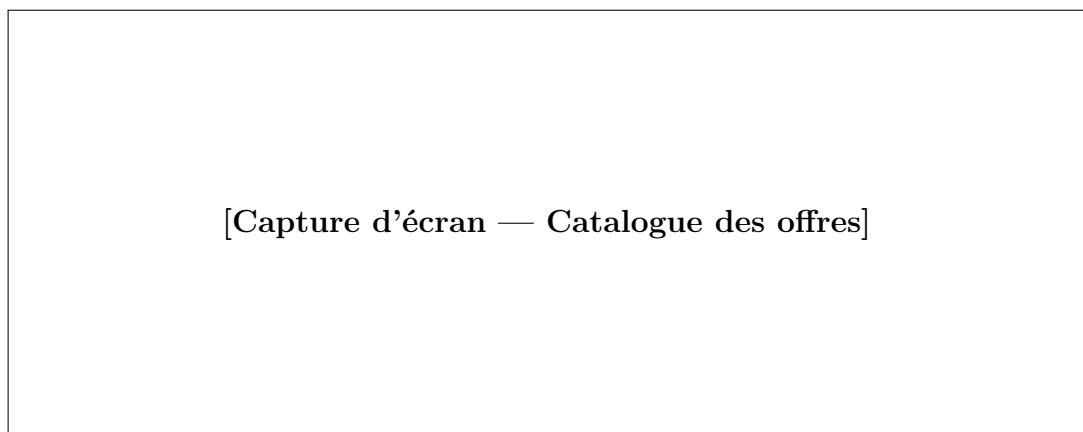


FIGURE 3.11 – Interface du catalogue des offres

3. Sprint 3 — Billing-service, Auth, API Gateway et Frontend

3.1 Objectif du sprint

Ce sprint finalise le développement métier avec le service de facturation, l’authentification JWT, le gateway centralisé et l’interface Angular.

3.2 Sprint Backlog

ID	Tâche	Priorité	Points
3.1	Génération de factures pour un abonnement	Must	13
3.2	Export de facture en PDF professionnel	Must	5
3.3	Envoi de facture par email via Brevo SMTP	Should	5
3.4	Authentification JWT avec contrôle RBAC	Must	5
3.5	API Gateway avec routage et filtrage JWT	Must	8
3.6	Interface Angular (boutiques, clients, abonnements, factures)	Must	13
3.7	Eureka Server pour la découverte des services	Should	3

TABLE 3.3 – Sprint Backlog — Sprint 3

3.3 Diagramme de cas d'utilisation — Sprint 3

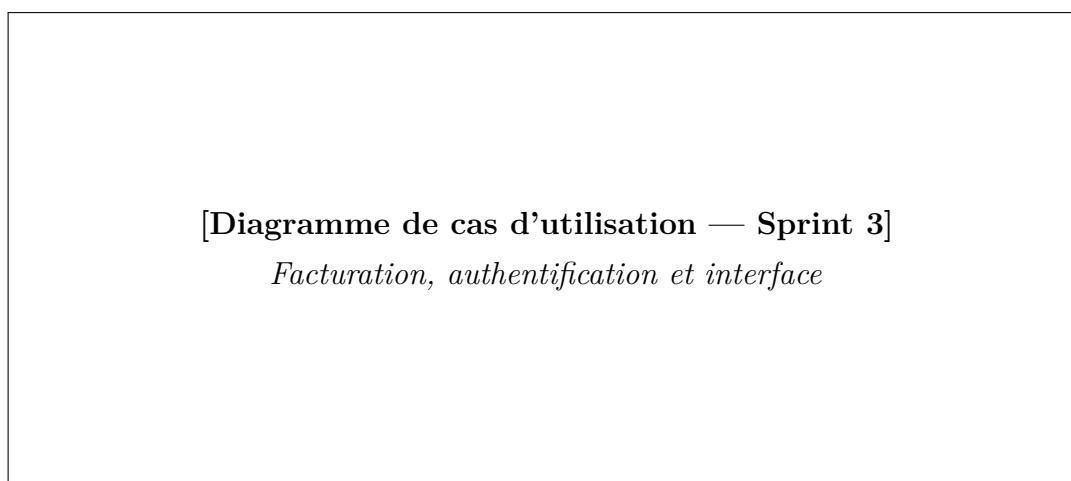


FIGURE 3.12 – Diagramme de cas d'utilisation du Sprint 3

3.4 Diagramme de classes — Sprint 3

3.4.1 Billing-service

Le microservice Billing gère le cycle de vie des factures :

- **Facture** : Contient le numéro de facture, la référence client, le montant HT, le taux de TVA (19%), le montant TTC, le statut (DRAFT, SENT, PAID, OVERDUE, CANCELLED) et les dates d'émission et d'échéance.
- **LigneFacture** : Représente une ligne dans la facture avec la description du service, la quantité, le prix unitaire et le montant.

[Diagramme de classes — Billing-service]
Facture, LigneFacture

FIGURE 3.13 – Diagramme de classes du Billing-service

3.4.2 Authentication-service

Le microservice Authentication gère l'accès sécurisé :

- **User** : Contient le nom d'utilisateur, le mot de passe chiffré, le rôle (ADMIN, RESPONSABLE_BOUTIQUE, AGENT_COMMERCIAL) et le statut du compte.

[Diagramme de classes — Authentication-service]
User, Role (enum)

FIGURE 3.14 – Diagramme de classes de l'Authentication-service

3.5 Diagramme de séquence — Générer une facture

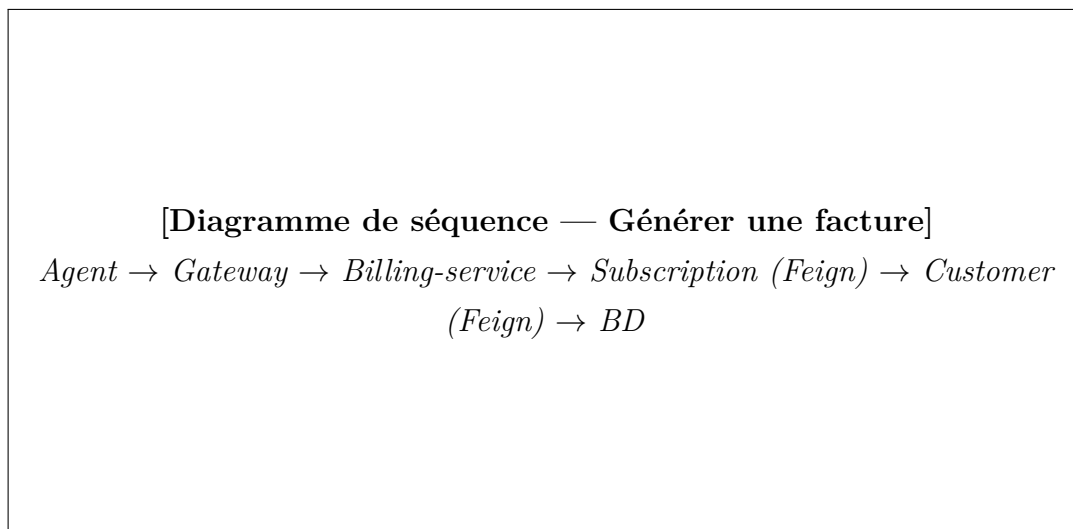


FIGURE 3.15 – Diagramme de séquence : génération d’une facture

3.6 Architecture technique de la Release 1

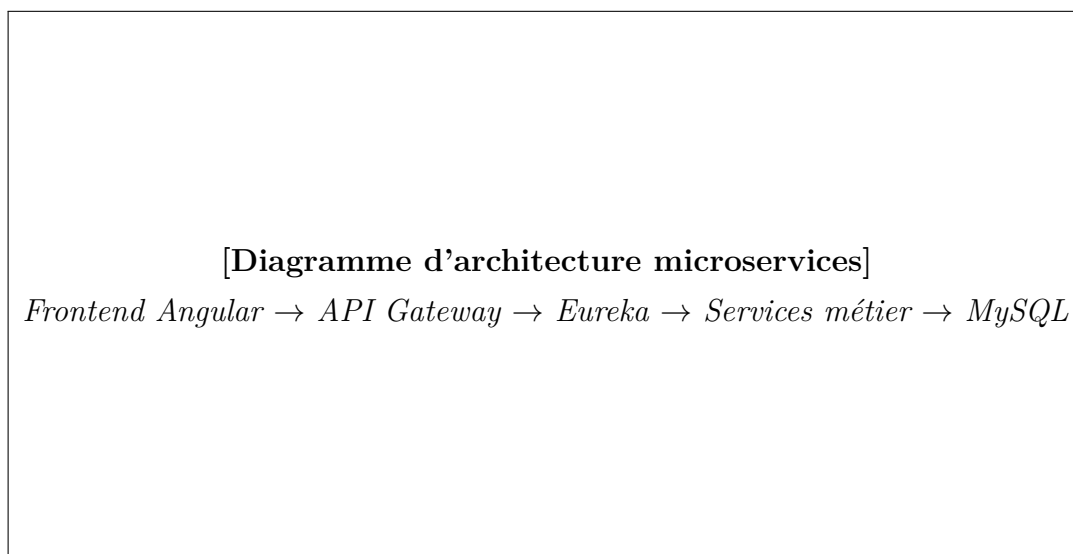
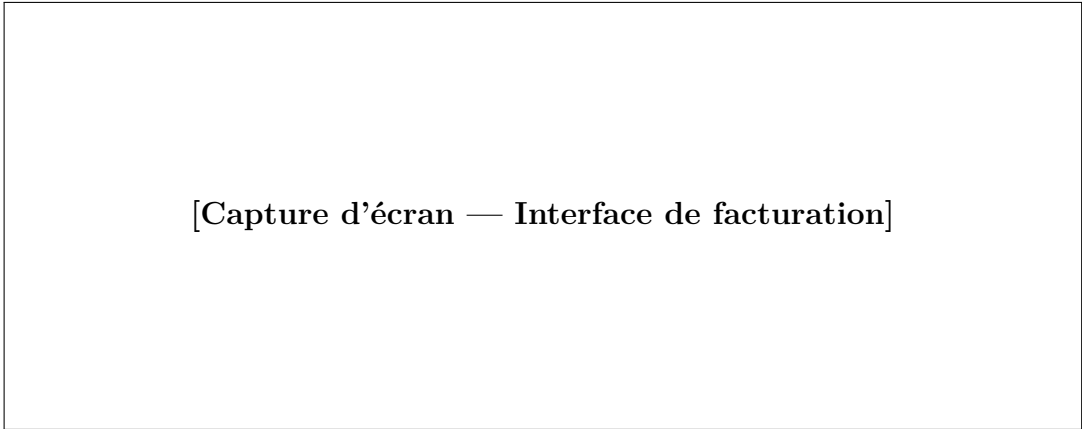


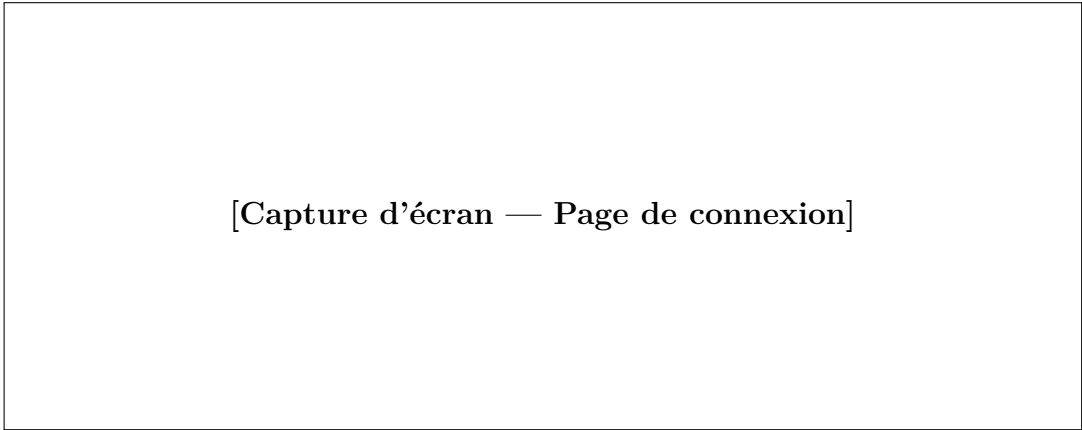
FIGURE 3.16 – Architecture technique des microservices

3.7 Interfaces réalisées — Sprint 3



[Capture d'écran — Interface de facturation]

FIGURE 3.17 – Interface de gestion des factures



[Capture d'écran — Page de connexion]

FIGURE 3.18 – Interface d'authentification

Conclusion

Au terme de cette release, l'ensemble des microservices métier sont opérationnels : gestion des boutiques, des clients, du catalogue, des abonnements et de la facturation. L'authentification JWT et l'API Gateway assurent la sécurité et le routage centralisé. Le chapitre suivant abordera la mise en place de l'infrastructure de déploiement.

Chapitre 4. Release 2 — Mise en place de l'infrastructure

Introduction

Cette release couvre la containerisation des microservices et leur déploiement, d'abord sur un cluster Kubernetes local (Minikube), puis sur l'infrastructure cloud AWS. Elle est répartie sur deux sprints.

1. Sprint 4 — Containerisation et déploiement local

1.1 Objectif du sprint

Containeriser l'ensemble des microservices avec Docker et valider leur déploiement sur un cluster Kubernetes local avec Minikube.

1.2 Sprint Backlog

ID	Tâche	Priorité	Points
4.1	Rédaction des Dockerfiles multi-étapes pour chaque microservice	Must	5
4.2	Rédaction du fichier docker-compose.yml pour l'environnement local	Must	5
4.3	Rédaction des manifests Kubernetes (Deployment, Service, ConfigMap)	Must	8
4.4	Déploiement et validation sur Minikube	Must	5

TABLE 4.1 – Sprint Backlog — Sprint 4

1.3 Containerisation avec Docker

Chaque microservice dispose d'un Dockerfile multi-étapes qui sépare la phase de compilation (build) de la phase d'exécution (runtime), permettant de réduire la taille de l'image finale.

```
1 # Etape 1 : Compilation
2 FROM maven:3.9-eclipse-temurin-21 AS build
```

```
3 WORKDIR /app
4 COPY pom.xml .
5 COPY src ./src
6 RUN mvn clean package -DskipTests
7
8 # Etape 2 : Execution
9 FROM eclipse-temurin:21-jre-alpine
10 COPY --from=build /app/target/*.jar app.jar
11 EXPOSE 8081
12 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Listing 4.1 – Exemple de Dockerfile multi-étapes

1.4 Orchestration locale avec Docker Compose

Le fichier `docker-compose.yml` orchestre l'ensemble des services en environnement de développement local : base de données MySQL, les microservices Spring Boot, l'API Gateway, le frontend Angular et le serveur Eureka.

1.5 Déploiement sur Minikube

Minikube fournit un cluster Kubernetes local pour valider les manifests avant le déploiement en production. Les manifests sont organisés en quatre répertoires :

- `infra/` : MySQL (StatefulSet avec PersistentVolumeClaim).
- `config/` : ConfigMap et Secrets pour les variables d'environnement.
- `services/` : Deployments et Services pour chaque microservice.
- `frontend/` : Deployment et Service pour l'application Angular.

[Capture d'écran — Pods sur Minikube]

```
kubectl get pods -n billing-dev
```

FIGURE 4.1 – Pods déployés sur le cluster Minikube

2. Sprint 5 — Déploiement sur le cloud AWS

2.1 Objectif du sprint

Provisionner l'infrastructure cloud AWS et déployer la plateforme en environnement de production.

2.2 Sprint Backlog

ID	Tâche	Priorité	Points
5.1	Provisionnement du cluster EKS via la console et CLI AWS	Must	8
5.2	Création de l'instance RDS MySQL	Must	5
5.3	Configuration du registre ECR pour les images Docker	Must	3
5.4	Adaptation des manifests K8s pour l'environnement cloud	Must	5
5.5	Premier déploiement sur EKS et validation	Must	5

TABLE 4.2 – Sprint Backlog — Sprint 5

2.3 Architecture de l'infrastructure AWS

L'infrastructure de production repose sur les services managés suivants :

- **Amazon EKS** : Cluster Kubernetes managé pour l'orchestration des conteneurs.
- **Amazon RDS** : Instance MySQL managée pour la persistance des données.
- **Amazon ECR** : Registre privé pour le stockage des images Docker.
- **Amazon EC2** : Instance hébergeant le serveur Jenkins pour le pipeline CI/CD.

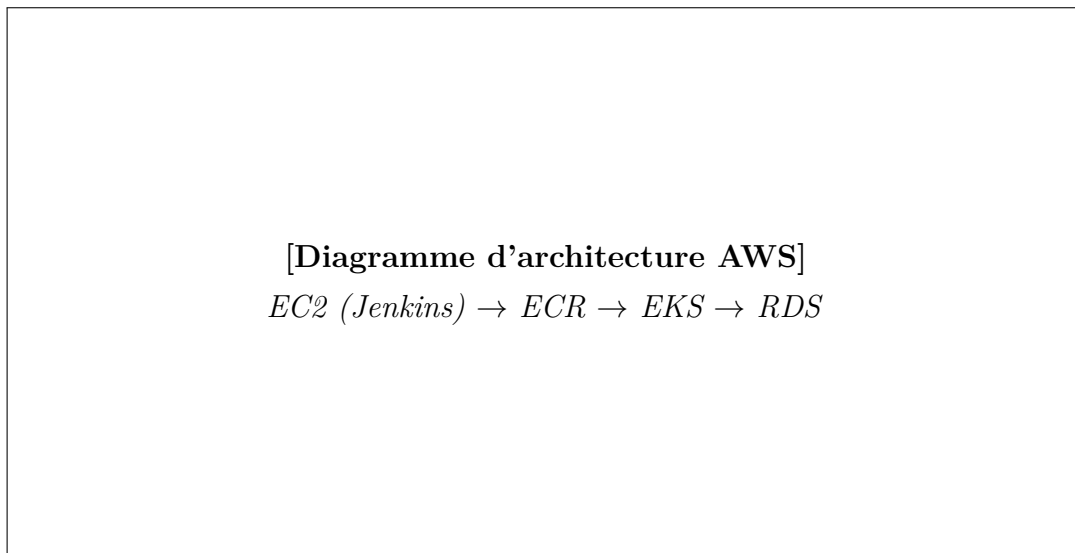


FIGURE 4.2 – Architecture de l'infrastructure AWS

2.4 Provisionnement du cluster EKS

Le cluster EKS a été provisionné via la console AWS et l'outil `eksctl`. La configuration inclut la création de groupes de nœuds (node groups) avec les ressources nécessaires pour l'exécution des microservices.

2.5 Configuration d'Amazon RDS

Une instance MySQL 8.0 a été créée sur Amazon RDS. Les bases de données suivantes sont provisionnées : `db_authentication`, `db_customer`, `db_catalog`, `db_subscription`, `db_boutique`, `db_billing`.

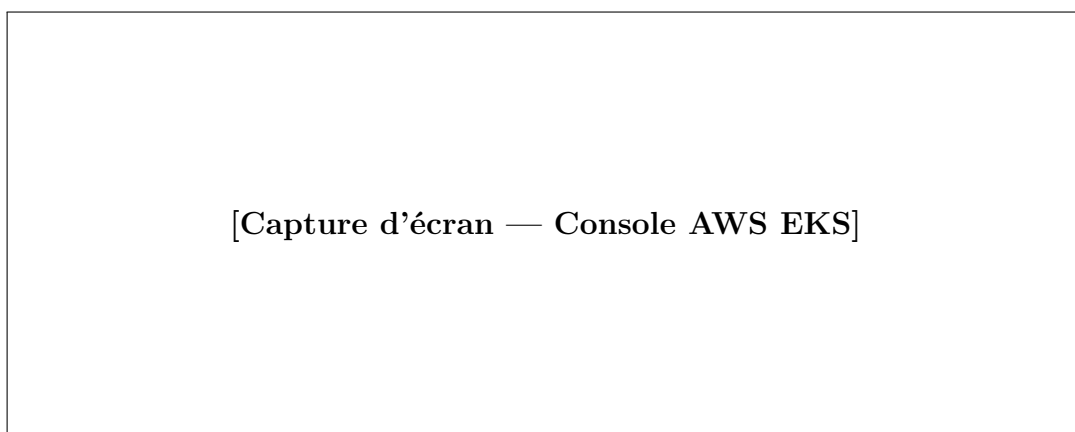


FIGURE 4.3 – Cluster EKS sur la console AWS

Conclusion

Au terme de cette release, l'infrastructure de déploiement est opérationnelle : les micro-services sont containerisés, validés localement sur Minikube, et l'environnement cloud AWS (EKS, RDS, ECR) est provisionné. Le chapitre suivant abordera l'automatisation du pipeline CI/CD.

Chapitre 5. Release 3 — Pipeline CI/CD

Introduction

Cette release vise à automatiser l'intégration et la livraison continues du projet. Elle couvre la mise en place du pipeline Jenkins, l'analyse de qualité avec SonarQube, le scan de sécurité avec Trivy, le packaging avec Helm et le déploiement continu avec ArgoCD.

1. Sprint 6 — Pipeline CI/CD et déploiement continu

1.1 Objectif du sprint

Mettre en place un pipeline d'intégration et de livraison continues complet, depuis le commit du code jusqu'au déploiement automatique sur le cluster EKS.

1.2 Sprint Backlog

ID	Tâche	Priorité	Points
6.1	Configuration du serveur Jenkins (Dockerfile custom)	Must	5
6.2	Pipeline multi-étapes : build Maven, tests unitaires	Must	8
6.3	Intégration SonarQube pour l'analyse de qualité du code	Should	5
6.4	Scan de sécurité des images avec Trivy	Should	3
6.5	Push automatisé des images vers AWS ECR	Must	5
6.6	Configuration d'ArgoCD et Helm pour le déploiement GitOps	Must	8

TABLE 5.1 – Sprint Backlog — Sprint 6

1.3 Architecture du pipeline CI/CD

Le pipeline CI/CD suit un modèle GitOps. Le processus complet se déroule en deux phases :

1. **Intégration continue (CI)** — gérée par Jenkins :
 - Checkout du code source depuis GitHub.

- Compilation avec Maven et exécution des tests unitaires.
- Analyse de la qualité du code avec SonarQube.
- Construction des images Docker.
- Scan de sécurité des images avec Trivy.
- Push des images vers AWS ECR.
- Mise à jour des tags d'images dans le dépôt de configuration GitOps.

2. **Livraison continue (CD)** — gérée par ArgoCD :

- ArgoCD surveille le dépôt `billing-app-config` (repo GitOps).
- Détection automatique des changements de tags d'images.
- Synchronisation automatique des manifests Kubernetes sur le cluster EKS.

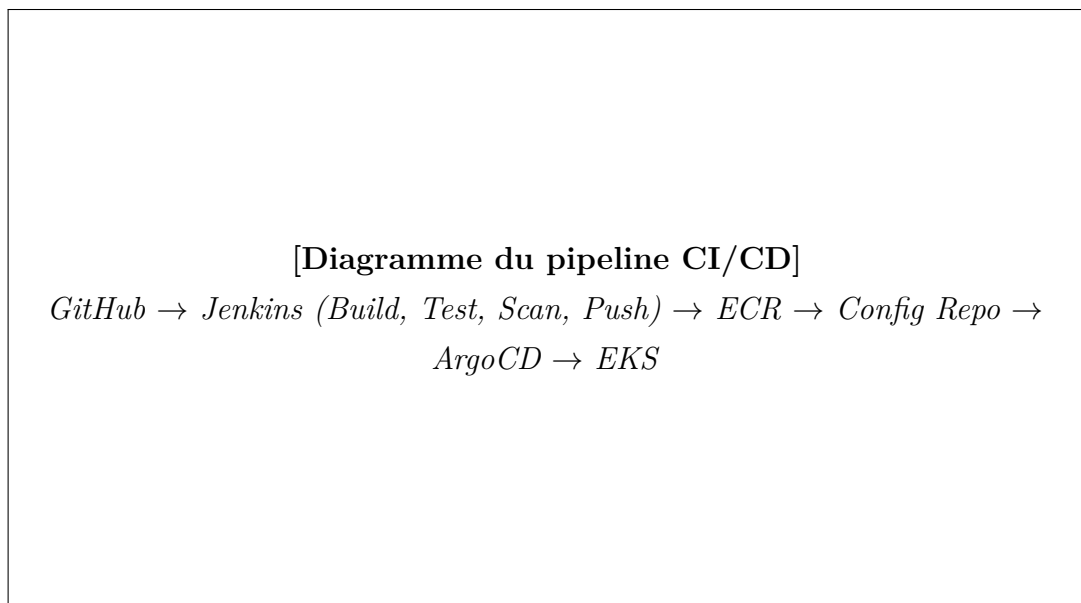


FIGURE 5.1 – Architecture du pipeline CI/CD

1.4 Serveur Jenkins

Le serveur Jenkins est déployé sur une instance EC2 dédiée. Son image Docker est personnalisée pour inclure tous les outils nécessaires :

```
1 FROM jenkins/jenkins:lts
2 USER root
3 RUN apt-get update && \
4     apt-get install -y docker.io git curl maven && \
5     apt-get clean
6 # Trivy
7 RUN curl -sL https://raw.githubusercontent.com/aquasecurity/
8     trivy/main/contrib/install.sh | sh -s -- -b /usr/local/bin
9 # Helm
```

```
10 RUN curl https://raw.githubusercontent.com/helm/helm/  
11     main/scripts/get-helm-3 | bash  
12 # AWS CLI  
13 RUN curl "https://awscli.amazonaws.com/awscli-exe-  
14 ubuntu_linux-x86_64.zip" -o "awscliv2.zip" && \  
15     unzip awscliv2.zip && ./aws/install
```

Listing 5.1 – Dockerfile Jenkins personnalisé

1.5 Pipeline Jenkinsfile

Le pipeline est défini dans un fichier `Jenkinsfile` déclaratif à la racine du projet.

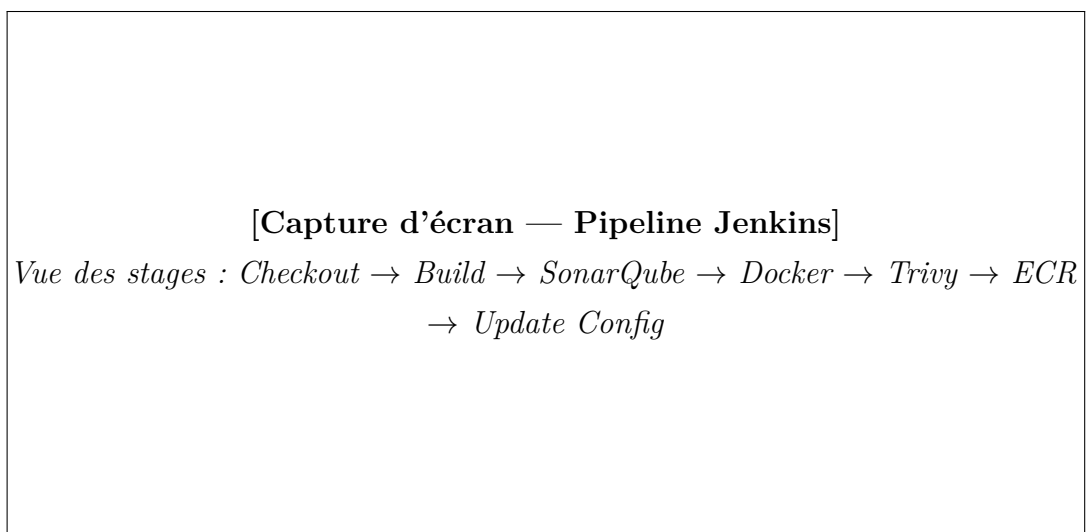


FIGURE 5.2 – Pipeline Jenkins avec les différentes étapes

1.6 Analyse de qualité — SonarQube

SonarQube est utilisé pour l'analyse statique du code. Il détecte les bugs, les vulnérabilités et les mauvaises pratiques de code (code smells).

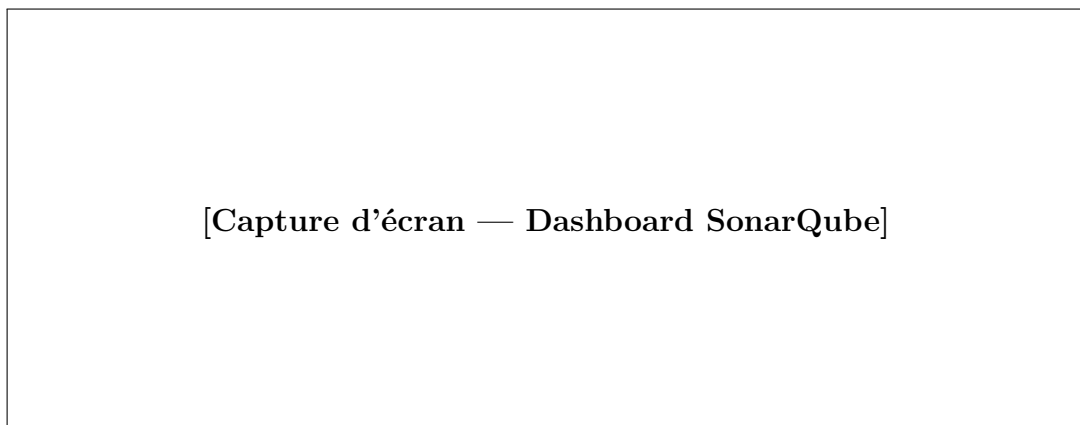


FIGURE 5.3 – Résultats d'analyse SonarQube

1.7 Scan de sécurité — Trivy

Trivy est un scanner de vulnérabilités pour les images Docker. Il est exécuté dans le pipeline Jenkins après la construction de chaque image.

1.8 Déploiement GitOps avec ArgoCD

ArgoCD surveille le dépôt de configuration `billing-app-config` et synchronise automatiquement les manifests Kubernetes sur le cluster EKS.

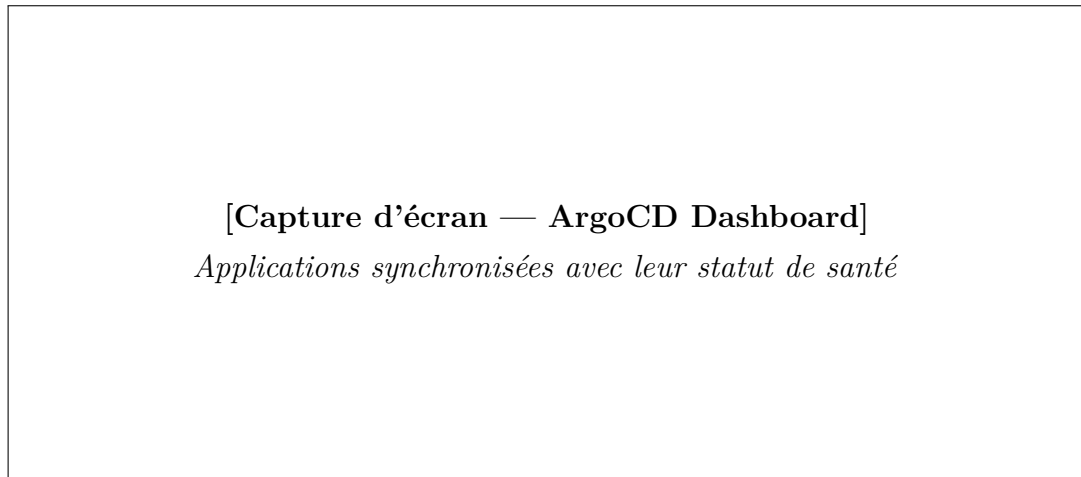


FIGURE 5.4 – Dashboard ArgoCD

1.9 Packaging avec Helm

Helm est utilisé pour le packaging des déploiements Kubernetes en production. Les Helm charts permettent de paramétrer les déploiements (image tags, replicas, variables d'environnement) via un fichier `values.yaml`.

Conclusion

Au terme de cette release, le pipeline CI/CD est pleinement opérationnel. Chaque commit déclenche automatiquement la compilation, les tests, l'analyse de qualité, le scan de sécurité et le déploiement sur le cluster EKS via ArgoCD. Le chapitre suivant abordera la mise en place du monitoring.

Chapitre 6. Release 4 — Monitoring et supervision

Introduction

Cette dernière release met en place la supervision des microservices déployés sur le cluster Kubernetes. L'objectif est de collecter les métriques des services, de les visualiser à travers des tableaux de bord et de configurer des alertes en cas d'anomalie.

1. Sprint 7 — Prometheus, Grafana et dashboards

1.1 Objectif du sprint

Déployer une stack de monitoring complète sur le cluster EKS pour superviser la santé et les performances des microservices en production.

1.2 Sprint Backlog

ID	Tâche	Priorité	Points
7.1	Déploiement de Prometheus sur le cluster Kubernetes	Should	3
7.2	Déploiement de Grafana et configuration des sources de données	Should	3
7.3	Création des dashboards de supervision des microservices	Should	5
7.4	Configuration des ServiceMonitors pour le scraping des métriques	Should	3

TABLE 6.1 – Sprint Backlog — Sprint 7

1.3 Architecture de monitoring

La stack de monitoring est composée de trois outils complémentaires :

- **Prometheus** : Système de collecte de métriques. Il interroge (scrape) périodiquement les endpoints `/actuator/prometheus` exposés par chaque microservice Spring Boot.

- **Grafana** : Plateforme de visualisation. Elle se connecte à Prometheus comme source de données et permet de créer des tableaux de bord interactifs.
- **Alertmanager** : Gestionnaire d'alertes. Il reçoit les alertes de Prometheus et les achemine vers les canaux de notification configurés.

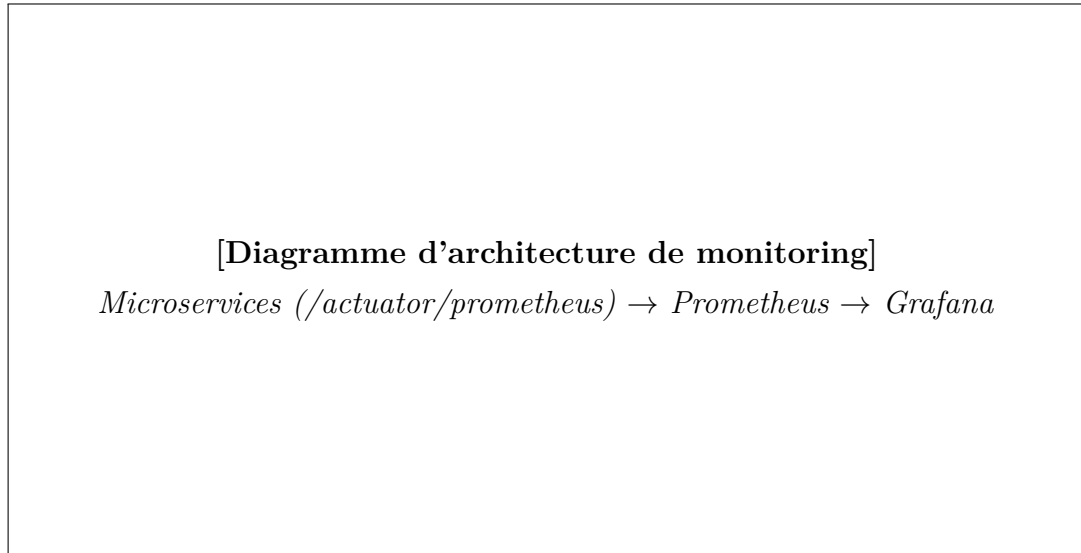


FIGURE 6.1 – Architecture de la stack de monitoring

1.4 Déploiement de Prometheus

Prometheus est déployé sur le cluster Kubernetes dans un namespace dédié `monitoring`. Les `ServiceMonitor` (ressources personnalisées) sont configurés pour cibler automatiquement les microservices portant le label `app.kubernetes.io/part-of: billing-platform`.

1.5 Dashboards Grafana

Plusieurs tableaux de bord ont été créés pour visualiser les métriques clés :

- **Vue globale** : Nombre de pods actifs, utilisation CPU et mémoire par service.
- **Requêtes HTTP** : Taux de requêtes, temps de réponse (p50, p95, p99) et taux d'erreurs par microservice.
- **JVM** : Utilisation du heap, nombre de threads actifs et activité du garbage collector.

[Capture d'écran — Dashboard Grafana]
Vue globale des microservices

FIGURE 6.2 – Dashboard Grafana de supervision

[Capture d'écran — Prometheus Targets]
Liste des endpoints scrapés

FIGURE 6.3 – Targets Prometheus

Conclusion

La mise en place de la stack de monitoring complète cette dernière release. L'ensemble de la plateforme de facturation télécom est désormais opérationnelle en production, avec une supervision continue des performance et de la santé des microservices.

Conclusion générale et perspectives

Ce projet de fin d'études a eu pour ambition la refonte architecturale et le déploiement automatisé d'un système de gestion et de facturation télécom (BSS). Face à la rigidité des systèmes monolithiques traditionnels, la solution proposée a permis de démontrer les bénéfices liés à l'adoption d'une architecture orientée microservices, combinée aux pratiques DevOps modernes.

Tout au long de ce projet, nous avons suivi une démarche agile et itérative, structurée en quatre releases majeures, garantissant une évolution maîtrisée du produit :

- **La conception logicielle** : Nous avons scindé les domaines métier en microservices distincts (Customer, Boutique, Catalog, Subscription, Billing et Auth), assurant ainsi l'indépendance du code et des bases de données.
- **L'orchestration et le Cloud** : Le déploiement s'est concrétisé par la containerisation Docker et la mise en place d'une infrastructure robuste sur Amazon EKS, illustrant la maîtrise des écosystèmes Cloud Native.
- **Culture DevOps et automatisation** : La conception d'un pipeline CI/CD complet via Jenkins, intégrant les tests de sécurité (Trivy), d'analyse de code (SonarQube) et le paradigme GitOps (ArgoCD), a démontré notre capacité à automatiser la chaîne de livraison logicielle.
- **Supervision** : Enfin, la mise en place de Prometheus et Grafana a clôturé le cycle de vie du projet en fournissant la visibilité nécessaire pour garantir la stabilité de l'application en production.

Ce projet a été riche en défis techniques et en apprentissages. Il nous a permis d'approfondir nos compétences en ingénierie logicielle (Java, Spring Boot, Angular), en architecture distribuée et en administration d'infrastructures cloud et Kubernetes.

Limites et Perspectives

Bien que la solution réponde aux objectifs initiaux de ce PoC (Proof of Concept), une prise de recul architecturale permet d'identifier certaines limites, représentant autant de perspectives d'amélioration pour une mise en production industrielle :

1. **Couplage temporel (Le "Monolithe Distribué")** : Actuellement, la communication inter-services repose sur des appels synchrones via OpenFeign (HTTP/REST). Ce choix pragmatique crée un couplage fort : si un service échoue, l'appel en cascade échoue également. La solution radicale pour l'avenir sera la transition vers une **Architecture Orientée Événements (EDA)** utilisant un broker de messages (Apache Kafka), garantissant l'asynchronisme et la haute résilience.

2. **Cohérence des données (Saga Pattern)** : En l'absence de transactions de bases de données globales, les erreurs partielles peuvent entraîner des incohérences. L'implémentation du patron **Saga** permettrait d'orchestrer des transactions distribuées via des requêtes de compensation.
3. **Gestion des performances et latence réseau** : Les multiples appels réseau nécessaires pour la facturation ralentissent le système. La mise en place du pattern **CQRS** (Command Query Responsibility Segregation) et d'un système de **Cache distribué** (Redis) pour des données peu volatiles (comme le catalogue d'offres) réduirait considérablement la latence, à condition de maîtriser l'invalidation de cache pour éviter toute erreur de tarification.

En définitive, ce projet de fin d'études constitue une fondation technique extrêmement solide, alignée sur les standards actuels de l'industrie informatique. Les perspectives identifiées prouvent que l'ingénierie logicielle est un processus d'amélioration continue, visant sans cesse à allier résilience opérationnelle et excellence technique.

Bibliographie

Webographie

Sites web consultés

1. PostgreSQL Documentation

<https://www.postgresql.org/docs/>

Consulté le : 8 mars 2025

Description : Documentation officielle et complète de PostgreSQL.

2. Docker Documentation

<https://docs.docker.com/>

Consulté le : 19 mars 2025

Description : Guide officiel pour la containerisation, les images et les déploiements.

3. Kubernetes Documentation

<https://kubernetes.io/docs/>

Consulté le : 5 mars 2025

Description : Documentation et meilleures pratiques pour l'orchestration de conteneurs.

4. Apache NiFi Documentation

<https://nifi.apache.org/documentation/>

Consulté le : 13 mars 2025

Description : Guides et référence de NiFi, outil d'orchestration de flux de données.

5. GitHub Actions Documentation

<https://docs.github.com/en/actions>

Consulté le : 22 mars 2025

Description : Documentation officielle sur l'automatisation CI/CD via GitHub Actions.

6. DevOps Best Practices (Azure DevOps)

<https://docs.microsoft.com/azure/devops/>

Consulté le : 10 mars 2025

Description : Guide Microsoft pour les pratiques DevOps incluant CI/CD.

7. ChatGPT (OpenAI)

<https://chatgpt.com/>

Consulté le : 3 mars 2025

Description : Assistant IA conversationnel basé sur GPT, pour la rédaction, la recherche et le codage.

8. Claude (Anthropic)

<https://claude.ai/>

Consulté le : 3 mars 2025

Description : Assistant IA alternatif axé sur la sécurité, la précision et la productivité.

9. **GitHub Copilot**

<https://github.com/features/copilot>

Consulté le : **3 mars 2025**

Description : Assistant IA de programmation pour l'autocomplétion et la génération de code.

10. **Metabase Documentation**

<https://www.metabase.com/docs/>

Consulté le : **12 mars 2025**

Description : Documentation officielle de Metabase pour la visualisation et l'analyse de données.

11. **Apache Spark Documentation**

<https://spark.apache.org/docs/latest/>

Consulté le : **15 mars 2025**

Description : Référence technique pour le traitement distribué et l'analytique big data.

12. **OWASP Top Ten**

<https://owasp.org/www-project-top-ten/>

Consulté le : **21 mars 2025**

Description : Guide des risques majeurs de sécurité applicative, utile pour la sécurisation des APIs et des plateformes BI.

13. **Data Engineering Podcast**

<https://www.dataengineeringpodcast.com/>

Consulté le : **18 mars 2025**

Description : Podcast sur les architectures, outils et retours d'expérience en data engineering.

14. **Allianz Group**

<https://www.allianz.com/en.html>

Consulté le : **2 mars 2025**

Description : Site institutionnel du groupe Allianz, pour la compréhension du contexte métier et des enjeux sectoriels.

Tutoriels et articles techniques

1. **Building Scalable Data Pipelines**

<https://medium.com/data-engineering>

Consulté le : **14 mars 2025**

Description : Articles techniques sur la conception de pipelines de données évolutifs.

2. **Start Data Engineering (blog)**

<https://www.startdataengineering.com/>

Consulté le : **25 mars 2025**

Description : Blog spécialisé sur les pratiques, outils et architectures en data engineering.

3. **Kubernetes Best Practices**

<https://cloud.google.com/kubernetes-engine/docs/best-practices>

Consulté le : **7 mars 2025**

Description : Bonnes pratiques pour la mise en œuvre de Kubernetes sur Google Cloud.

4. **Database Design Patterns (PostgreSQL)**

<https://www.postgresql.org/docs/current/ddl.html>

Consulté le : **17 mars 2025**

Description : Patterns et bonnes pratiques pour la conception de schémas relationnels.

5. **API Design Guidelines (REST)**

<https://restfulapi.net/>

Consulté le : **11 mars 2025**

Description : Directives pour la conception, la normalisation et la sécurisation des APIs REST.

6. **Modern Data Stack (Fivetran Blog)**

<https://fivetran.com/blog/modern-data-stack>

Consulté le : **16 mars 2025**

Description : Analyse des tendances et outils de la data stack moderne.

7. **DataOps Manifesto**

<https://www.dataopsmanifesto.org/>

Consulté le : **20 mars 2025**

Description : Principes et bonnes pratiques pour l'industrialisation des pipelines de données.

8. **YouTube – Apache NiFi Tutorials (DataCouch)**

<https://www.youtube.com/playlist?list=PLf0swTFhTI8q18p6U6QvFqQnQKp1K0QdC>

Consulté le : **9 mars 2025**

Description : Tutoriels vidéo pour la prise en main et l'automatisation de flux avec NiFi.

9. **Towards Data Science (Medium)**

<https://towardsdatascience.com/>

Consulté le : **23 mars 2025**

Description : Articles de vulgarisation et d'approfondissement sur la data science, l'IA et l'ingénierie des données.

10. **Awesome Data Engineering (GitHub)**

<https://github.com/igorbarinov/awesome-data-engineering>

Consulté le : **27 mars 2025**

Description : Liste collaborative de ressources, outils et lectures recommandées pour l'ingénierie des données.

11. **Le Monde Informatique – Dossiers Data**

<https://www.lemondeinformatique.fr/dossiers/data/>

Consulté le : **28 février 2025**

Description : Dossiers et actualités sur la transformation digitale, la gouvernance et la valorisation des données en entreprise.

12. **Qwen (Alibaba Cloud)**

<https://qwen.alibaba.com/>

Consulté le : **4 mars 2025**

Description : Modèle d'IA générative open source développé par Alibaba Cloud, utilisé pour la génération de texte et l'assistance à la rédaction technique.

13. **Gemini (Google AI)**

<https://deepmind.google/technologies/gemini/>

Consulté le : **8 mars 2025**

Description : Modèle d'IA multimodal de Google, utilisé pour la recherche, la génération de code et l'analyse de données complexes.

14. **Google Cloud Copilot**

<https://cloud.google.com/ai/copilot>

Consulté le : **12 mars 2025**

Description : Assistant IA de Google Cloud pour l'aide à la programmation, la génération de code et l'automatisation des workflows cloud.